Crystallographic Lattice Classification with Deep Learning and MagmaDNN

Keh, Sedrick
sedrickkeh@gmail.com

Nichols, Daniel
dnicho22@vols.utk.edu

Chan, Kam Fai
kfchan1998@gmail.com

July 2019

Abstract

Every material can be associated with 1 of 230 possible crystallographic space groups. This paper explores the task of crystal lattice structure classification from given electron diffraction image intensity patterns. The proposed solution uses several deep learning and computer vision methodologies to implement the classification. More specifically, the ResNet model is examined in closer detail. Additionally, this paper also highlights the MagmaDNN framework and discusses how it can be applied to the given microscopy problem. This application demonstrates MagmaDNN's flexibility and efficiency. Promising results in both speed and accuracy are discussed.

Contents

1 Introduction

Recent advancements in deep learning have led to numerous breakthroughs in various fields. One such noteworthy area where deep learning applications have made a significant impact is microscopy. In microscopy, the effectiveness of such neural approaches is further enhanced by the vast amounts of data available from empirical observations and experiments.

Putting this into context, Computer Vision (CV) has been a particularly active field that has both benefited largely from deep learning advancements and contributed immensely to their growth. The analysis of images (together with the various tasks associated with it) lends itself very well to these deep neural network architectures.

To trace this growth trajectory, one can simply observe the progress of the ILSVRC ImageNet classification challenge [1] throughout the years. The main breakthrough began in 2012, when AlexNet [2] successfully achieved then-unprecedented performance by using deep convolutional networks. The years that followed saw continuous improvements, with models such as GoogLeNet [3], VGGNet [4], and ResNet[5] each introducing novel techniques to improve network performance on image classification tasks. Subsequently, various unsupervised architectures and generative models have also been rapidly emerging, the most prominent of which is the Generative Adversarial Network (GAN) [6], which has shown great promise in multiple image generation tasks. Many of these aforementioned architectures and networks can also be easily modified for tasks such as object detection, facial recognition, super-resolution, and many others.

1.1 Deep Learning for Microscopy

Computational microscopy, or imaging on a very small scale, is a subset of computer vision that uses computational techniques to process image data from microscopes and other micro-scale imaging tools. Deep learning is also very effective in microscopy tasks, similar to how it is very effective with regular computer vision applications. However, there are some challenges associated especially with computational microscopy that may not be so evident in other imaging tasks. For instance, some errors in images may carry over from measurement errors from using the microscopy tools, leading to biased data. Additionally, unlike normal images of objects such as dogs or cars, the images in microscopy may not be that easy to interpret to a user who is not familiar with traditional microscopy.

Tasks in microscopy include classification, super-resolution, energy predictions, simulations, and many others. In this paper, the main task of focus is the classification of crystallographic lattice space groups.

1.2 Crystallographic Space Groups

A material can be associated with its corresponding space group. This refers to its 3-dimensional configuration in space [7]. Taking into account the symmetries across groups, researchers have discovered that there are only a finite number of space groups. In total, there are 219 types; this becomes 230 when chiral copies are considered distinct. Below are some examples of basic lattice structures for crystals.

1.3 Task

The task is to classify each material into its corresponding space group. The inputs will be images from convergent beam electron diffraction (CBED) image intensities, and the target output is the correct appropriate class. More details can be found in section 1.4 below.

1.4 Dataset

1.4.1 HDF5

The data is stored in a hierarchical (HDF5) [8] format. Each HDF5 file can be thought of as a database that contains the information needed and some metadata describing the entire database. The main building block of the HDF5 format is the group, similar to folders in a directory. Under this group, there are datasets, which contain the main data themselves. Additionally, each dataset can have corresponding attributes, which is analogous to the dataset's metadata. These attributes can also give various features related to the dataset.

1.4.2 Dataset Overview

In the CBED image dataset[1], there around around 500 groups, each with around 400 datasets. Each dataset contains 1 input data item, which consists of a stack of a 3 by

[1]Data taken from Smoky Mountain Data Challenge 2019.

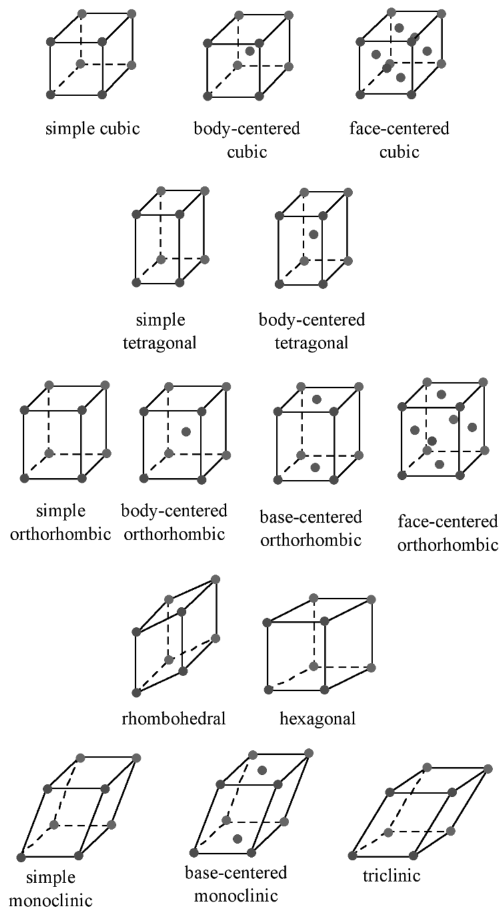

Figure 1: Examples of Symmetric Lattice Structures

512 by 512 input image, the corresponding output class (space group), together with some other features described below in Figure 2 and in the table in Figure 3.

The input data is in the form of images with dimensions 3 by 512 by 512, where 512 represents the width and height pixel dimensions, while the 3 channels represent three different crystallographic directions or material orientations of the lattice.

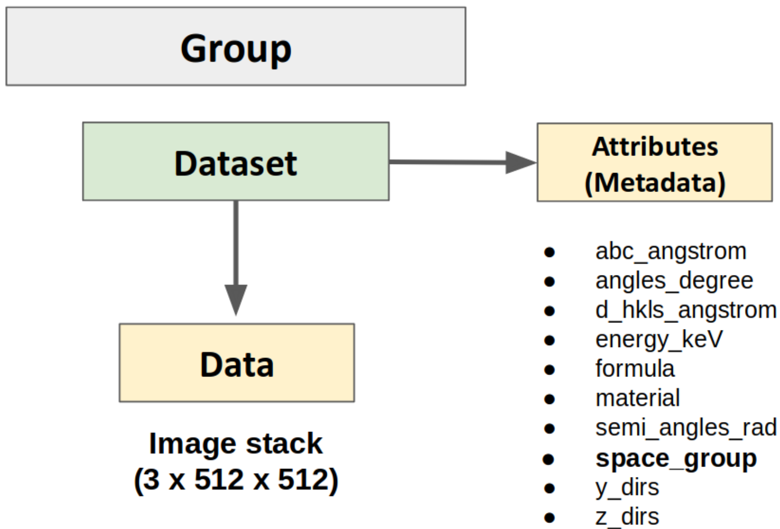

Figure 2: Structure of HDF5

The output data (space group) is represented by a number between 1 and 230, representing the corresponding lattice group label. In addition to the images and the space groups, the other features are also important even if they are not directly related to the inputs or outputs. This is because they can also be used in training the model, in a process commonly known as multitask learning [9]. In this research, however, multitask learning is not considered. Rather, the model only considers the input images and output classes.

1.4.3 Data Examples

Image intensities can range from 10^{-20} to 10^{-1}, where smaller values represent darker areas. Visualizing these intensities would reveal a general dark area with small bright spots in the middle, as shown in Figure 4 below.

In this image of unscaled data, most of the surrounding areas are covered in black (except the few bright spots). This is because compared to 10^{-3}, there is not much difference between a 10^{-12} and a 10^{-20} (both are very small). To address this issue, once can scaling the input data by raising all the pixels to a certain exponent < 1. In this case, an exponent factor power of 0.25 is used, and visualizing this reveals a more meaningful figure, as shown in Figure 5 below.

Feature	Type	Description
abc_angstrom	128-bit floating-point	lattice constants $\{a, b, c\}$ of the material (in Å)
angles_degree	128-bit floating-point	lattice angles $\{\alpha, \beta, \gamma\}$ of the material (in degrees)
d_hkls_angstrom	128-bit floating-point	the d-spacing of the Bragg reflections from which CBED was acquired (in Å)
energy_keV	64-bit integer	electron beam energy
formula	String, len = variable	chemical formula of material
material	String, len = 9	material name (as catalogued in `https://materialsproject.org/`)
semi_angles_rad	128-bit floating-point	half-angles of convergence of the incident electron beam (in radians)
space_group	String, len $\in 1, 2, 3$	crystallographic lattice structure group
y_dirs	128-bit floating-point	crystallographic (hkl) transverse directions of material
z_dirs	64-bit integer	crystallographic (hkl) normal directions of material

Figure 3: Description of Features

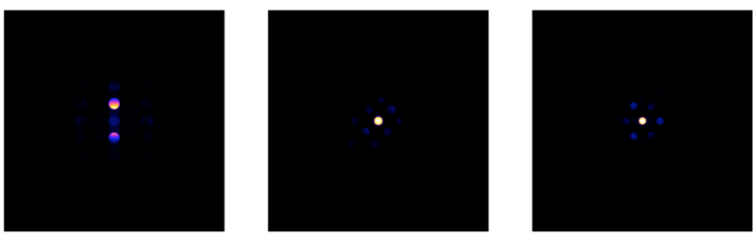

Figure 4: Unscaled Data Example: 3 by 512 by 512

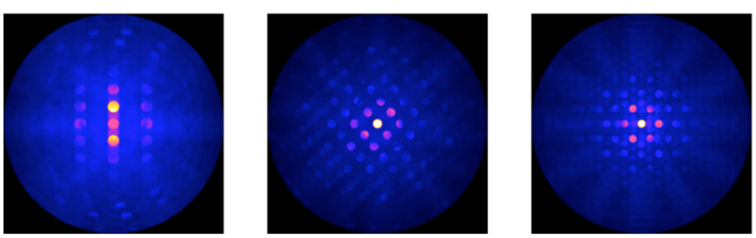

Figure 5: Scaled Data

1.5 Prior Work

There have been some previous work on using deep learning for crystal classification. Several of these works, however, have slight variations from the desired task: either the input data is of a different type or dimension (not from electron diffraction image intensities) or the output data prediction is not of the same space group class (but rather some other type of lattice classification).

For instance, a study on powder X-ray diffraction data using deep neural networks [10] revealed that convolutional neural networks, even shallow networks, yield relatively satisfactory performance on crystal classification. Other studies have also investigated the use of active learning in building such models [11]. For model interpretability, it is possible to use attentive response maps [12] to understand what is happening inside the black box model.

Another important aspect of these computational microscopy applications is their preprocessing methodologies, which adapt from certain traditional microscopy ideas. For instance, one possible aspect would be to cut the data into parts [13]. Another study [14] considered using some signal processing techniques to transform the input images for the model to learn more easily.

2 ResNet

This section will examine the ResNet[5] model, discussing its motivation, structure, and how it can be applied to this particular task (crystal lattice classification).

2.1 Degradation Problem

The main problem that ResNet is trying to tackle is the phenomenon when accuracy decreases as the number of layers becomes too large. This problem is known as the degradation problem and is caused by the function space becoming larger and thus harder to search for the optimal function. The figure below highlights how a 56 layer network actually performs worse than a shallower 20 layer network.

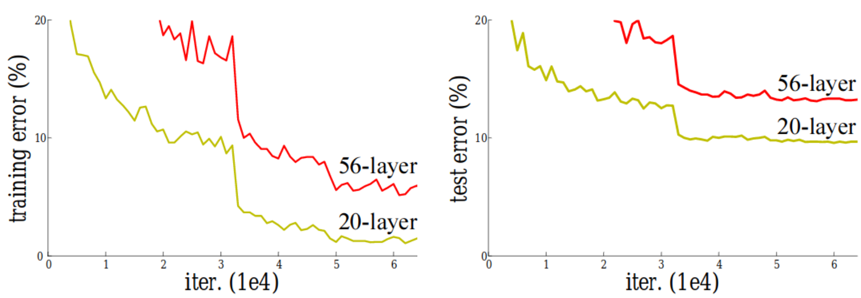

Figure 6: Degradation Problem

2.2 Skip Connections

The solution ResNet[5] introduces is something called a skip connection or shortcut connection, directly mapping the input x to the output. Thus, suppose that a group of neural network layers can be thought of as a function $F(\cdot)$. Then, under the ResNet structure, the output will become $F(x) + x$ instead of simply $F(x)$ (as in a normal neural network).

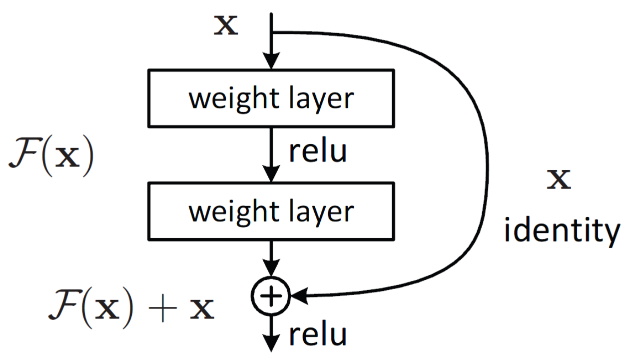

Figure 7: ResNet Skip Connection

2.3 ResNet Architectures

In the original ResNet paper[5], various ResNet architecture were introduced, all using the same residual block as its building block, but differing in the number of layers they include. The most popular residual networks are ResNet-18, ResNet-34, ResNet-50, and ResNet-102. The number of layers of these networks is indicated by the number appened to their names.

Furthermore, the paper introduced the idea of bottleneck layers, which uses 1 by 1 convolutions at the start and end of each residual block to project the input to a smaller dimension and project back the output to a larger dimension. This reduces the number of parameters in the main filters and decreases training time.

Further architecture details can be found in Appendix A.

3 MagmaDNN

MagmaDNN [15] is a modularized deep learning framework that is optimized for parallel computation and distributed training for GPUs. It currently supports most of the basic features for deep learning operations, including forward and backward propagation, dense layers, convolutional neural networks, pooling, dropout, and batch normalization. Other features include a custom dataloader, graph convolutions [16] and custom optimizers (SGD, AdaGrad, RMSProp, Adam[17]). The sections below detail some relevant features of MagmaDNN that are relevant to the computational microscopy task at hand.

3.1 Accelerated GPU Computations

MagmaDNN is built on various other toolkits that help with the GPU parallelism and optimization. It uses MAGMA[18] for highly optimized linear algebra computations. This ensures that such computations are as efficient as possible, maximizing the use of parallelism.

Additionally, for several deep learning-related applications, MagmaDNN is build on CuDNN functions. These include the forward and backward propagation for 2D convolutions, pooling operations, dropout operations, and others. Similar to MAGMA, CuDNN operations are also very highly optimized. For instance, convolutions are carried out using FFTs and the Winograd algorithm.

3.2 Custom Memory Manager

MagmaDNN defines its own memory manager, similar to that of CUDA. In MagmaDNN, this is called as "MANAGED" memory, similar to the "CUDA_MANAGED" memory offered by CUDA. The benefit of defining its own memory manager is increased efficiency for transfer in batches between CPU and GPU, which normally would have a very high overhead.

Aside from making memory operations more efficient, abstracting the memory manager into a single custom defined class also has the additional benefit of flexibility, allowing development to be much more customizable. This is in like with MagmaDNN's core goals.

3.3 Distributed Training

MagmaDNN currently supports MPI capabilities, allowing for both data parallelism and model parallelism. Data parallelism involves splitting up the data into various processing units then integrating them together at the end. Meanwhile, model parallelism involves splitting up different parts of the same model into various processing units. Using MagmaDNN to do data and model parallelism massively speeds up the training and would be very relevant for a task such as this, where the size of the dataset is very large and would normally take a lot of time if computed in serial.

Additionally, MAGMA and CuDNN alo exploit the state-of-the-art parallelism techniques, making the internal operations of MagmaDNN much more efficient.

4 Model

The model used is a modified version of ResNet-18. This begins with 16 channels, and doubles and downsamples after every two blocks. The structure of the model is shown below.

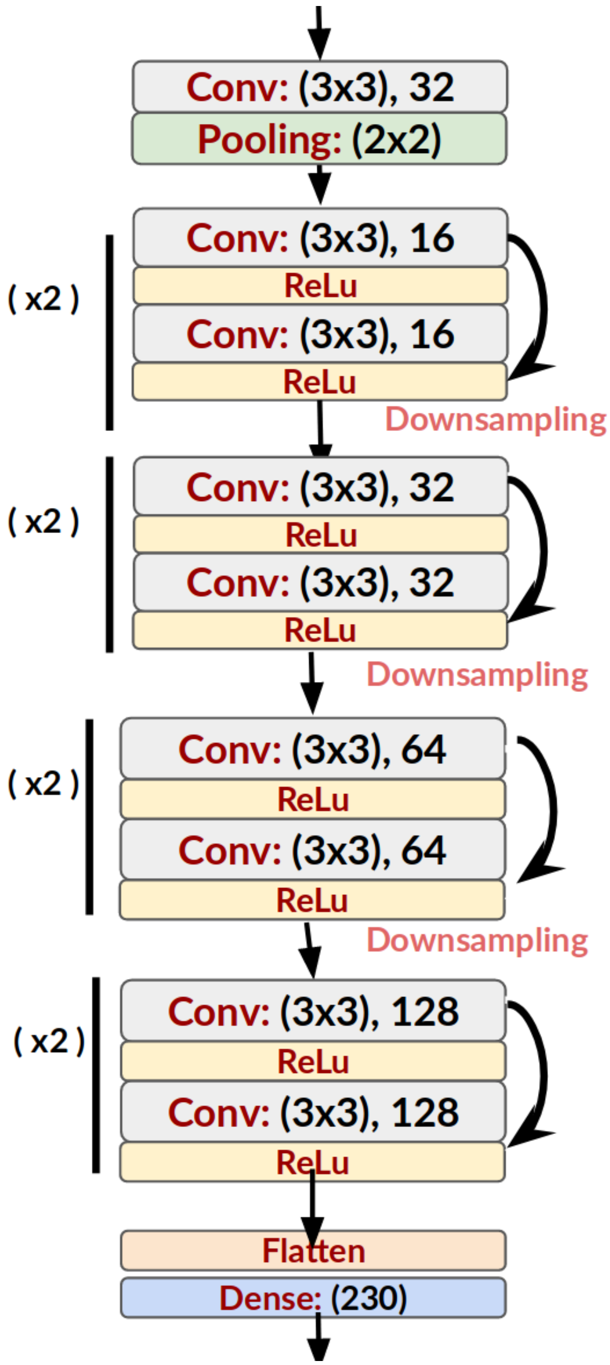

Figure 8: Modified ResNet-18

5 Results

5.1 Accuracy

Using a modified version of the ResNet-18 model discussed in section 2.3, the files were loaded one by one and trained for 5 epochs. With a learning rate of 0.001, batch size of 32, and using the Adam optimizer, the model was able to reach a training accuracy of 16% after training 9000 images.

Although this accuracy does not quite match to the current state-of-the-art accuracy of around 40%, it provides a substantial improvement over a random guess of 0.4% (picking 1 out of 230 possible classes). Furthermore, the main goal of this research was not to get the best possible accuracy, but rather to optimize the speed and efficiency of training using MagmaDNN. These will be further discussed in section 5.3.

5.2 Challenges

This section investigates some possible explanations of why the model performs poorly on this particular dataset.

1. **Large number of output classes**: There are 230 total classes for classification. Furthermore, an initial overview of some of the data shows that a lot of these 230 classes actually appear quite similar to each other, indicating that it may be difficult for the model to discern between two classes that look very similar.

2. **Big Data**: There are around 500 files, each containing around 400 images. In total, the data takes around 500GB. Because the data is so large, given the resources and time available, this research only considered a small subset of the data, training only for 9000 images.

3. **Data Imbalance**: In total there are 230 classes to classify into, representing the 230 different space groups naturally occurring in nature. However, some of these 230 groups are very rare and not found very often. Reflecting this, the data, which was generated from a simulation, only contained around 200 out of the 230 possible classes, meaning that 30 classes never appeared at all in the training data. Furthermore, a handful of other classes only appeared once or twice. This data imbalance problem makes the model worse and biased towards predicting classes that appear more often.

5.3 MagmaDNN Efficiency and Scalability

The most important part of the research was whether MagmaDNN is feasible to use for these types of problems, and whether MagmaDNN can scale to problems involving very large amounts of data.

Benchmarks using the ResNet have proven this quite successful. For one epoch, MagmaDNN finishes it in an average of 195 seconds, as opposed to TensorFlow's average of 726 seconds, more than a three-times speedup. This is reflected in the table in the next page:

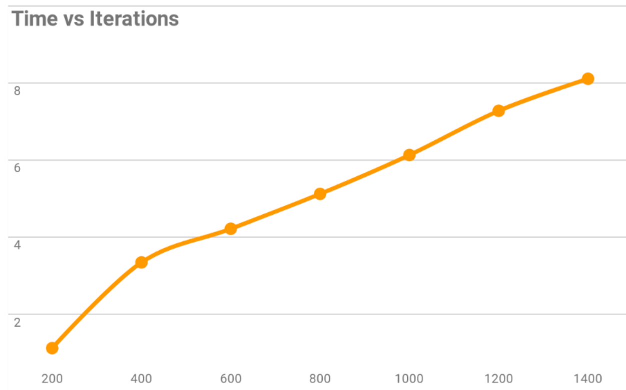

Figure 9: MagmaDNN Scalability

6 Conclusion

This paper demonstrates the effectiveness of MagmaDNN when applied to regular data science tasks, more specifically, computational microscopy. MagmaDNN's flexible modularity allows for ease in building new features such as ResNet. It also successfully uses parallel computation to outperform other deep learning frameworks in terms of speed and efficiency.

7 Future Work

Work can be done in two fronts, namely the task-specific crystal classification side and the MagmaDNN core development.

7.1 Microscopy

The main problem to be solved is the data imbalance problem. One solution to solving this may be to consider certain data augmentation techniques. For instance, it is possible to resample from the sparse classes in order to increase the number of occurrences.

One other possible area to look into would be multitask prediction. In this project, only the input images and output classes were considered in the prediction. However, previous research has shown that multitask learning can in fact improve how the model learns certain features of the inputs. In this specific problem, multitask learning can be applied by further considering the other classes (*angles*, *y_dirs*, *z_dirs*, etc.) as outputs to be predicted.

7.2 MagmaDNN

Parallelism can also be further incorporated into MagmaDNN to improve the speed and efficiency of the implementation. Currently, there is also a memory manager improvement that is in progress, moving towards using smart pointers and references to make MagmaDNN easier to use and faster.

8 Availability

MagmaDNN is open-sourced. The MagmaDNN v1.0 release, together with succeeding releases and updates, is hosted at BitBucket[2].

Acknowledgements

Research was done with support from the National Science Foundation, Oak Ridge National Laboratory, Joint Institute of Computational Sciences, University of Tennessee, Knoxville, Hong Kong University of Science and Technology, and Chinese University of Hong Kong. This work would not be possible without assistance and mentorship provided by supervisors Dr. Kwai Wong, Dr. Stanimire Tomov, and Dr. Junqi Yin.

References

[1] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision (IJCV)*, 115(3):211–252, 2015.

[2] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems 25*, pages 1097–1105. Curran Associates, Inc., 2012.

[3] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Computer Vision and Pattern Recognition (CVPR)*, 2015.

[4] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *CoRR*, abs/1409.1556, 2014.

[5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015.

[6] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems 27*, pages 2672–2680. Curran Associates, Inc., 2014.

[7] Howard Hiller. Crystallography and Cohomology of Groups. *The American Mathematical Monthly*, 93:765–779, December 1986.

[8] The HDF Group. Hierarchical data format version 5, 2000-2010.

[2] `https://bitbucket.org/icl/magmadnn`

[9] Sebastian Ruder. An overview of multi-task learning in deep neural networks. *CoRR*, abs/1706.05098, 2017.

[10] Jung J Sohn K Singh SP Pyo M Shin N Sohn KS Park WB, Chung J. Classification of crystal structure using a convolutional neural network. *IUCrJ*, 4:486–494, 2017.

[11] Evgeny V. Podryabinkin, Evgeny V. Tikhonov, Alexander V. Shapeev, and Artem R. Oganov. Accelerating crystal structure prediction by machine-learning interatomic potentials with active learning. *Phys. Rev. B*, 99:064114, Feb 2019.

[12] Angelo Ziletti, Devinder Kumar, Matthias Scheffler, and Luca Ghiringhello. Insightful classification of crystal structures using deep learning. *Nature Communications*, 9, 2018.

[13] Volker L. Deringer, Davide M. Proserpio, Gábor Csányi, and Chris J. Pickard. Data-driven learning and prediction of inorganic crystal structures. *Faraday Discuss.*, 211:45–59, 2018.

[14] Pascal Marc Vecsei, Kenny Choo, Johan Chang, and Titus Neupert. Neural network based classification of crystal symmetries from x-ray diffraction patterns. *Phys. Rev. B*, 99:245120, Jun 2019.

[15] Daniel Nichols, Nathalie-Sofia Tomov, Frank Betancourt, Stanimire Tomov, Kwai Wong, and Jack Dongarra. Magmadnn: Towards high-performance data analytics and machine learning for data-driven scientific computing. *MAGMA*, 60(80):100, 2019.

[16] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *CoRR*, abs/1609.02907, 2016.

[17] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2014. cite arxiv:1412.6980Comment: Published as a conference paper at the 3rd International Conference for Learning Representations, San Diego, 2015.

[18] Stanimire Tomov, Jack Dongarra, and Marc Baboulin. Towards dense linear algebra for hybrid GPU accelerated manycore systems. *Parallel Computing*, 36(5-6):232–240, June 2010.

Appendix A ResNet architectures

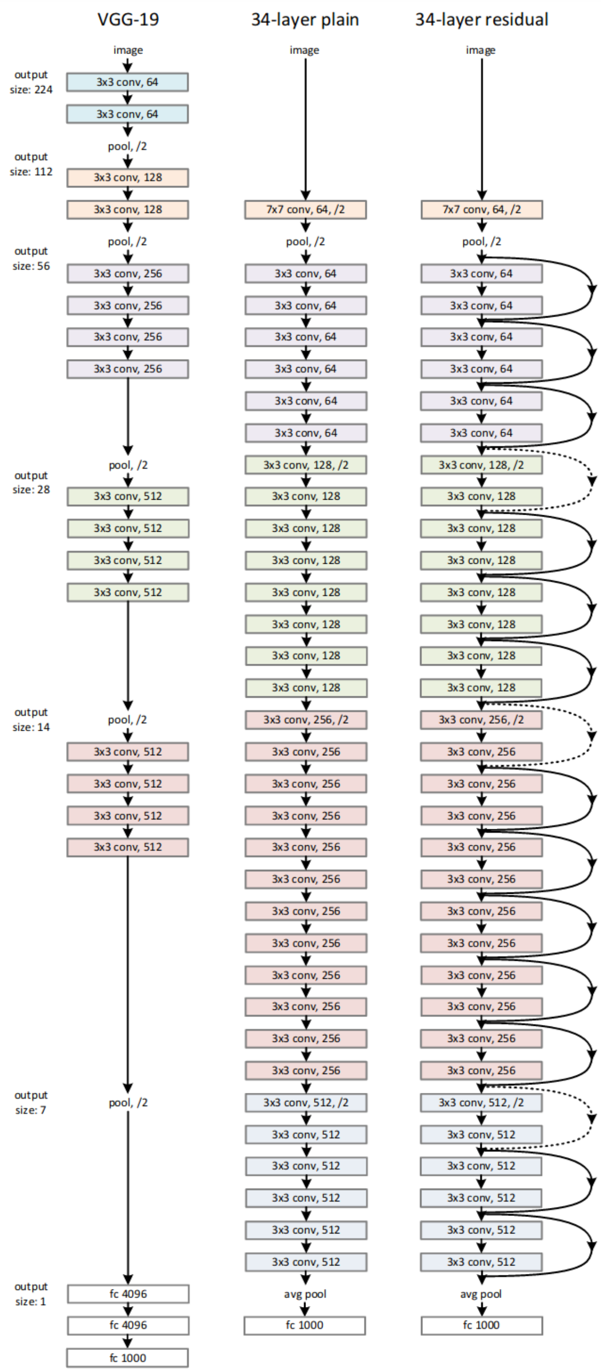

Figure 10: ResNet Architecture